740. Delete and Earn

You are given an integer array nums. You want to maximize the number of points you get by performing the following operation any number of times:

- Pick any nums[i] and delete it to earn nums[i] points. Afterwards, you must delete **every** element equal to nums[i] - 1 and **every** element equal to nums[i] + 1.

Return *the **maximum number of points** you can earn by applying the above operation some number of times*.

Code:

```
class Solution {
    public int deleteAndEarn(int[] nums) {

        int max = 0;

        for (int num : nums) {
            max = Math.max(max, num);
        }

        int[] points = new int[max + 1];

        for (int num : nums) {
            points[num] += num;
        }

        int prev2 = 0;
        int prev1 = 0;

        for (int i = 0; i <= max; i++) {

            int current = Math.max(
                prev1,
                prev2 + points[i]
            );

            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }
}
```

416. Partition Equal Subset Sum

Given an integer array nums, return true *if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or* false *otherwise*.

Code:

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for(int num : nums)
            sum+=num;
        if(sum%2!=0) return false;
        int target = sum/2;
        int n = nums.length;
        boolean[] dp = new boolean[target+1];
        if(nums[0]<=target)
            dp[nums[0]]=true;
        dp[0]=true;

        for(int i =1;i<n;i++){
            boolean[] curr = new boolean[target+1];
            curr[0] = true;
            for(int j =1;j<=target;j++){
                boolean np = dp[j];
                boolean p = false;
                if(nums[i]<=j){
                    p = dp[j-nums[i]];
                }
                curr[j]=p||np;
            }
            dp=curr;
        }


        return dp[target];
    }
}
```

494. Target Sum

You are given an integer array nums and an integer target.

You want to build an **expression** out of nums by adding one of the symbols '+' and '-' before each integer in nums and then concatenate all the integers.

- For example, if nums = [2, 1], you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1".

Return the number of different **expressions** that you can build, which evaluates to target.

Code:

```java
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = 0;
        for (int num : nums)
            sum += num;
        if ((sum + target) % 2 != 0)
            return 0;
        target = Math.abs(target);
        int t = (sum + target) / 2;
        int[] dp = new int[t + 1];
        if (nums[0] <= t)
            dp[nums[0]] = 1;

        dp[0] = 1;

        if (nums[0] == 0)
            dp[0] = 2;

        for (int i = 1; i < nums.length; i++) {
            int[] curr = new int[t+1];
            for (int j = 0; j <= t; j++) {
                int np = dp[j];
                int p = 0;
                if (nums[i] <= j)
                    p = dp[j - nums[i]];
                curr[j] = p + np;
            }
            dp=curr;
        }
        return dp[t];
    }
}
```

983. Minimum Cost For Tickets

You have planned some train traveling one year in advance. The days of the year in which you will travel are given as an integer array days. Each day is an integer from 1 to 365.

Train tickets are sold in **three different ways**:

- a **1-day** pass is sold for costs[0] dollars,
- a **7-day** pass is sold for costs[1] dollars, and
- a **30-day** pass is sold for costs[2] dollars.

The passes allow that many days of consecutive travel.

- For example, if we get a **7-day** pass on day 2, then we can travel for 7 days: 2, 3, 4, 5, 6, 7, and 8.

Return *the minimum number of dollars you need to travel every day in the given list of days*.

Code:

```
class Solution {
    public int mincostTickets(int[] days, int[] costs) {

        boolean[] travel = new boolean[366];
        for (int day : days) {
            travel[day] = true;
        }

        int[] dp = new int[366];

        for (int day = 1; day <= 365; day++) {
            if (!travel[day]) {
                dp[day] = dp[day - 1];
            }
            else {

                int oneDay = dp[day - 1] + costs[0];

                int sevenDay = dp[Math.max(0, day - 7)]
                             + costs[1];

                int thirtyDay = dp[Math.max(0, day - 30)]
                              + costs[2];

                dp[day] = Math.min(
                    oneDay,
                    Math.min(sevenDay, thirtyDay)
                );
            }
        }

        return dp[365];
    }
}
```

455 Alex doesn't like boredom. That's why whenever he gets bored, he comes up with games. One long winter evening he came up with a game and decided to play it.

Given a sequence a consisting of n integers. The player can make several steps. In a single step he can choose an element of the sequence (let's denote it a_k) and delete it, at that all elements equal to $a_k + 1$ and $a_k - 1$ also must be deleted from the sequence. That step brings a_k points to the player.

Alex is a perfectionist, so he decided to get as many points as possible. Help him.

Code:

```java
import java.util.*;

public class Main{
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        int n = in.nextInt();
        int[] a = new int[n];
        HashMap<Integer,Integer> freq = new HashMap<>();
        int max =0;
        for(int i =0;i<n;i++){
            a[i]=in.nextInt();
            freq.put(a[i],freq.getOrDefault(a[i],0)+1);
            max = Math.max(max,a[i]);
        }

        long dp[] = new long[max+1];
        dp[0]=0;
        if (max >= 1) {
            dp[1] = (long)1 * freq.getOrDefault(1, 0);
        }
        for(int i =2 ;i<=max;i++){
            long take = (long)i*freq.getOrDefault(i,0)+dp[i-2];
            long notTake = dp[i-1];
            dp[i]=Math.max(take,notTake);
        }
        System.out.println(dp[max]);
    }
}
```